

Ex. No. : 05

Date:

Register No.: 231701014

Name: N GOKUL KRISHNA

A Program To Implement Multi Layer Perceptron With Back Propagation

Aim:

To implement a **Multi Layer Perceptron with Backpropagation** to classify **Diabetes with low risk and high risk**

Requirements:

Jupyter Notebook with Python, creditcard.csv dataset, and required Python libraries.

PROGRAM:

Step 1: Import Libraries

```
import pandas as pd
import numpy as np
```

```
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

Step 2: Load Dataset

```
data = load_diabetes(as_frame=True).frame
data.head()
```

	age	sex	bmi	bp	s1	s2	s3	s4	s5	s6	target
0	0.038076	0.050680	0.061696	0.021872	-0.044223	-0.034821	-0.043401	-0.002592	0.019907	-0.017646	151.0
1	-0.001882	-0.044642	-0.051474	-0.026328	-0.008449	-0.019163	0.074412	-0.039493	-0.068332	-0.092204	75.0
2	0.085299	0.050680	0.044451	-0.005670	-0.045599	-0.034194	-0.032356	-0.002592	0.002861	-0.025930	141.0
3	-0.089063	-0.044642	-0.011595	-0.036656	0.012191	0.024991	-0.036038	0.034309	0.022688	-0.009362	206.0
4	0.005383	-0.044642	-0.036385	0.021872	0.003935	0.015596	0.008142	-0.002592	-0.031988	-0.046641	135.0

Step 3: Define Features and Target

0 → Low Risk
1 → High Risk

```
data['Class'] = (data['target'] > data['target'].mean()).astype(int)
```

```
X = data.drop(["target", "Class"], axis=1)  
y = data["Class"]
```

Step 4: Feature Scaling

```
scaler = StandardScaler()  
X_scaled = scaler.fit_transform(X)
```

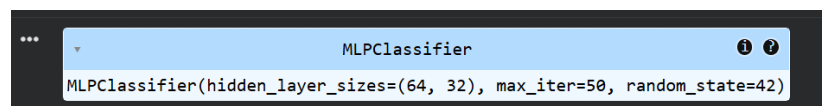
Step 5: Split Dataset

```
X_train, X_test, y_train, y_test = train_test_split(X, y,  
                                                    test_size=0.2, random_state=42  
)
```

Step 6: Train MLP Model (Backpropagation happens internally)

```
model = MLPClassifier(  
    hidden_layer_sizes=(64, 32), # two hidden layers  
    activation='relu',  
    solver='adam',  
    max_iter=50,  
    random_state=42  
)
```

```
model.fit(X_train, y_train)
```



Step 7: Predict Results

```
y_pred = model.predict(X_test)
```

Step 8: Model Evaluation

Accuracy

```
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.7303370786516854
```

Confusion Matrix

```
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
Confusion Matrix:
[[43  7]
 [17 22]]
```

Classification Report

```
print("Classification Report:")
print(classification_report(y_test, y_pred))
```

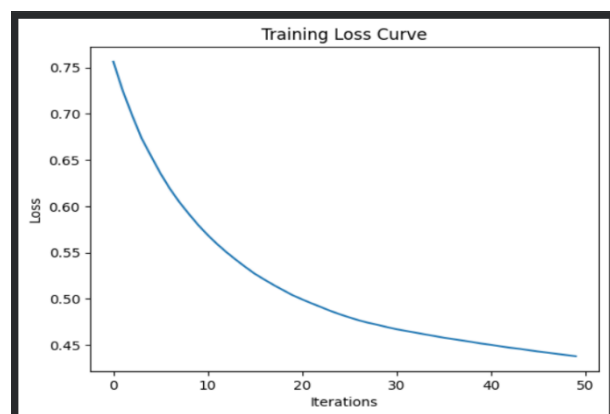
```
Classification Report:
              precision    recall  f1-score   support

     0           0.72         0.86         0.78         50
     1           0.76         0.56         0.65         39

   accuracy          0.73
  macro avg          0.74         0.71         0.71         89
 weighted avg          0.74         0.73         0.72         89
```

Loss Curve

```
import matplotlib.pyplot as plt
plt.plot(model.loss_curve_)
plt.title("Training Loss Curve")
plt.xlabel("Iterations")
plt.ylabel("Loss")
plt.show()
```



Result:

Thus, the Multi Layer Perceptron with Backpropagation was successfully implemented and evaluated using the dataset.